

C02_AT1_CSA0302 - Analytical based Question

Name: Deepak R

Register Number: 192511008

Course Code: CSA0302

Course Name: Data Structures

Question 1

Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately.

Linear Queue:

In an array-based linear queue, the rear index only moves forward. Even if dequeue operations free spaces at the beginning, those positions cannot be reused. After several alternating enqueue and dequeue operations, the rear eventually reaches the end of the array, resulting in a "Queue Full" condition although empty slots exist. This wastes memory and may require shifting elements, reducing efficiency.

Circular Queue:

A circular queue treats the array as circular. When the rear reaches the last index, it wraps around to the beginning if free positions are available. During 100 alternating enqueue and dequeue operations, freed positions are reused efficiently, preventing false overflow. Both enqueue and dequeue operations remain $O(1)$, and memory utilization is much better.

Comparison:

- Time Complexity: Both enqueue and dequeue are $O(1)$.
- Memory Usage: Linear queue wastes space; circular queue reuses space efficiently.
- Overflow: Linear queue may report false overflow; circular queue avoids it until the queue is actually full.
- Overall Performance: Circular queue performs better for repeated alternating operations.

Question 2

A queue implemented using an array reports "Queue Full" even though some positions are empty.

(a) Analyze why this occurs.

In a linear queue, the front index moves forward after dequeue operations, but the rear index never moves backward. As a result, empty positions created at the beginning of the array cannot be reused. When the rear reaches the last array index, the implementation reports "Queue Full" despite having unused spaces at the front. This problem is known as **false overflow**.

(b) Explain how a circular queue solves this problem.

A circular queue connects the last array position to the first using modulo arithmetic. After dequeuing elements, the rear wraps around to reuse the vacant positions at the beginning of the array. Thus, all available spaces are utilized efficiently, false overflow is eliminated, and enqueue/dequeue operations continue in $O(1)$ time.